Homework #2

RELEASEDATE: 2026/03/31

DUEDATE:2026/04/14, 23:59 on iLearning 3.0

1. *請將作業繳交至 iLearning 3.0*
2. *可使用 C 或 C++*
3. *請對每題產生答案的檔案，命名為 ans 題號 (例如: 第一題，請產生 ans1.txt 檔案)*
4. *每題的程式碼檔案請命名為 DS 題號_學號 (例如: 學號為 7114056138，第一題程式碼檔名命名為 DS1_7114056138.c)*
5. *如對作業有任何疑問，請依照以下聯絡方式詢問助教*
6. *所有題目都有給定一個 linked list 的程式碼，請一定要使用給定的程式碼操作，否則該題以 0 分計算*

Teaching Assistant:

- Name: 黃意丞
- Email: g114056138@mail.nchu.edu.tw
- Lab: WCCCLab (S901)

1 Sort Chain (pt. 25)

You will be given N numbers and a positive integer M. You will have to sort the N numbers in ascending order of their modulo M value.

- If there is a tie between an odd number and an even number (that is their modulo M value is the same) then the odd number will precede the even number.
- If there is a tie between two odd numbers then the larger odd number will precede the smaller odd number
- If there is a tie between two even numbers then the smaller even number will precede the larger even number.

For remainder value of negative numbers follow the rule of C programming language.

```
struct ListNode {
    int val;
    struct ListNode *next;
};
```

Input

The input file contains several sets of inputs. Each set starts with two integers N ($0 < N \leq 10000$) and M ($0 < M \leq 10000$) which denotes how many numbers are within this set. Each of the next N lines contains one number each. These numbers should all fit

in 32-bit signed integer. Input is terminated by a line containing two zeroes.

Output

For each set of input produce N + 1 lines of outputs. The first line of each set contains the value of N and M. The next N lines contain N numbers, sorted according to the rules mentioned above. Print the last two zeroes of the input in the output file also.

Sample Input

15 3

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

0 0

Sample Output

15 3

15

9

3

6

12

13

7

1

4

10

11

5

2

8

14

0 0

2 Continued Fractions (pt. 25)

Let $b_0, b_1, b_2, \ldots, b_n$ be integers with $b_k > 0$ for k > 0.

The continued fraction of order n with coeficients $b_1, b_2, \ldots,$ b_n and the initial term b_0 is defined by the following expression

$$b_0 + \frac{1}{b_1 + \frac{1}{b_{2+\ldots+\frac{1}{b_n}}}}$$

which can be abbreviated as $[b_0; b_1, \ldots, b_n]$.

Write a program that determines the expansion of a given rational number as a continued fraction. To ensure uniqueness, make $b_n > 1$.

```
typedef struct Node {
    // you can add data type you want
    struct Node *next;
} Node;

typedef struct {
    // you can add data type you want
    Node *top;
} Stack;
```

Input

The first line will have a number N ($0 < N \leq 100$) with the number of test cases. The input consists of an undetermined number of rational numbers. Each rational number is defined by two integers, numerator and denominator.

Output

For each rational number given in the input, you should output the corresponding continued fraction.

Sample Input

2

43 19

1 2

Sample Output

[2;3,1,4]

[0;2]

3 Team Queue (pt. 25)

In a team queue each element belongs to a team. If an element enters the queue, it first searches the queue from head to tail to check if some of its teammates (elements of the same team) are already in the queue. If yes, it enters the queue right behind them. If not, it enters the queue at the tail and becomes the new last element. Dequeuing is done like in normal queues: elements are processed from head to tail in the order they appear in the team queue. Your task is to write a program that simulates such a team queue.

```
typedef struct Node {
    // you can add data type you want
    struct Node *next, *pre;
} Node;

typedef struct Queue {
    // you can add data type you want
    Node *front, *rear;
} Queue;
```

Input

The input file will contain one or more test cases. Each test case begins with the number of teams t ($1 \leq t \leq 1000$). Then t team descriptions follow, each one consisting of the number of elements belonging to the team and the elements themselves. Elements are integers in the range 0 ~ 999999. A team may consist of up to 1000 elements. Finally, a list of commands follows.

There are three different kinds of commands:

- ENQUEUE x — enter element x into the team queue
- DEQUEUE — process the first element and remove it from the queue
- STOP — end of test case

The input will be terminated by a value of 0 for t.

Output

For each test case, for each 'DEQUEUE' command, print the element which is dequeued on a single line. Print a blank line after each test case, even after the last one.

Sample Input

1

3 101 102 103

3 201 202 203

ENQUEUE 101

ENQUEUE 201

ENQUEUE 102

ENQUEUE 202

ENQUEUE 103

ENQUEUE 203

DEQUEUE

DEQUEUE

DEQUEUE

DEQUEUE

DEQUEUE

DEQUEUE

STOP

0

Sample Output

101

102

103

201

202

203

4 Discard the Number (pt. 25)

Given a sequence $\{x_1, x_2, \ldots, x_m\}$, where all numbers in the sequence are positive integers. Perform the following operation on this sequence: start by looking at x_1, then x_2, and continue in this manner. After reaching x_m, start again from x_1,

repeating this process cyclically. Each time you look at a number, say x_i, remember it and compare it with the last remembered number x_j. If the two numbers satisfy gcd(x_i, x_j) = 1, discard x_i from the sequence and forget it. Then, restart the process from x_{i+1}. This means that after discarding a number, you cannot immediately discard the next number in the subsequent step.

Your task is to determine which numbers will be discarded and output them in the order they are discarded.

```
struct Node {
    int order, num;
    struct Node *next, *prev;
};

struct LinkedList {
    struct Node *head;
};
```

Input

The first line of input gives the number of testcases, T(1≤T≤50). Then T test cases follow each described in the following way. The first line contains a single integer m (1≤m≤5×10^4) indicates the number of elements. The second line contains the integers

x_1, x_2, ..., x_m ($1 \le x_i \le 10^9$) separated by spaces, which indicate the number on elements.

Output

For each input produce one line of output. First, print a single integer k, which indicates the number of discarded elements. After that print k distinct integer: discarded elements in the order of their deletion.

Sample Input

5

5

5 9 2 10 15

6

1 2 4 2 4 2

2

1 2

1

1

1

2

Sample Output

2 2 3

2 2 1

2 2 1

1

1

0

Submission File:

請繳交一個壓縮檔，檔名為 DS_學號 (例如: 學號為 7114056138，則壓縮檔名為 DS_7114056138.zip)，將檔案上傳至 iLearning 3.0，內容包含

- 所有題目的程式碼
- README.txt，如果有任何疑問或建議